

Sentinel AI Security Audit Report

Audit Metadata

- **Filename:** safe.exe
- **SHA-256 Checksum:** 1637a023d61251962b138feb3cf8dd880f90b3e00e857f12c11a24fb0e22a80d
- **Verdict:** SAFE
- **Risk Level:** LOW
- **Total Functions Evaluated:** 18

Executive Summary

The Sentinel AI Security Audit has completed a deep-tissue binary analysis of `safe.exe`. This assessment utilized a Graph Neural Network (GNN) compiler wrapper to evaluate the structural and semantic integrity of the executable's machine code.

The audit focused on identifying structural patterns indicative of memory corruption vulnerabilities, specifically targeting Stack-based Buffer Overflows (CWE-121) and Out-of-bounds Writes (CWE-787). Our analysis confirms that all 18 evaluated function Control Flow Graphs (CFGs) conform to secure coding baselines. No anomalous branching, unsafe pointer arithmetic, or deprecated library calls (such as `gets`) were detected. The binary demonstrates robust boundary enforcement and adheres to the SEI CERT C coding standards, specifically regarding array bounds and pointer validation (ARR30-C).

Analysis Metrics

The following parameters define the scope and rigor of the static analysis performed:

- **Analysis Engine:** GNN-Based Semantic Feature Extraction.
- **Evaluation Scope:** Full Binary Static Analysis (18/18 Functions).
- **Heuristic Depth:** Control Flow Graph (CFG) Analysis, Data Flow Tracking, and Symbol Table Verification.
- **Average Confidence Score:** 99.4%
- **Compliance Mapping:** CWE (Common Weakness Enumeration) & SEI CERT C Standards.

Detailed Findings Section

Status: No security flaws or rule violations were flagged.

The internal logic of the binary was scrutinized for patterns that represent common exploit vectors. The GNN model compared the assembly-level instructions against known vulnerability signatures. All functions passed the inspection, showing no evidence of unchecked `memcpy` operations, lack of null-terminators, or stack-smashing opportunities.

Verified Safe Functions

The following function offsets/symbols were analyzed and found to be compliant with secure coding practices:

- ``_init_term`` (Safe)
- ``_main_entry`` (Safe)
- ``sub_401020`` (Safe)
- ``sub_401050`` (Safe)
- ``sub_4010A0`` (Safe)
- ``sub_401100`` (Safe)
- ``sub_401140`` (Safe)
- ``sub_401190`` (Safe)
- ``sub_401200`` (Safe)
- ``sub_401230`` (Safe)
- ``sub_401280`` (Safe)
- ``sub_4012C0`` (Safe)
- ``sub_401310`` (Safe)
- ``sub_401350`` (Safe)
- ``sub_4013A0`` (Safe)
- ``sub_401400`` (Safe)
- ``sub_401440`` (Safe)
- ``sub_401490`` (Safe)

****General Mitigations & Best Practices****

While `safe.exe` is currently rated as **SAFE**, maintaining a high security posture requires continuous adherence to modern defensive compilation and deployment strategies.

1. Compiler-Level Protections

To ensure future iterations of this software remain resilient against memory corruption, the following flags should be enforced during the build process:

- **Stack Canaries (`-fstack-protector-all`)**: Injects "guard" values before the return address to detect stack smashing.
- **Address Space Layout Randomization (ASLR)**: Ensures that memory addresses (stack, heap, libraries) are randomized, making exploit payloads difficult to land.
- **Data Execution Prevention (DEP/NX)**: Marks data segments as non-executable to prevent the execution of shellcode injected into buffers.

2. Secure Coding Standards (C/C++)

- **Boundary Validation**: Always validate the length of input data before copying it into a fixed-size buffer.
- **Safe API Usage**: Replace deprecated functions with their safer counterparts.
- ***Avoid***: ``strcpy``, ``strcat``, ``gets``.
- ***Use***: ``strncpy``, ``strncat``, ``fgets``.
- **Control Flow Integrity (CFI)**: Implement CFI to ensure that the program execution follows a predetermined path, preventing return-oriented programming (ROP) attacks.

3. Continuous Security Testing

- **Fuzz Testing**: Implement automated fuzzing (e.g., AFL++ or libFuzzer) to provide malformed inputs to the 18

functions to ensure no edge-case crashes occur.

- **Static Analysis (SAST):** Integrate Sentinel AI or similar GNN-based auditors into the CI/CD pipeline to catch regressions before deployment.

End of Report

Generated by Sentinel AI Security Systems